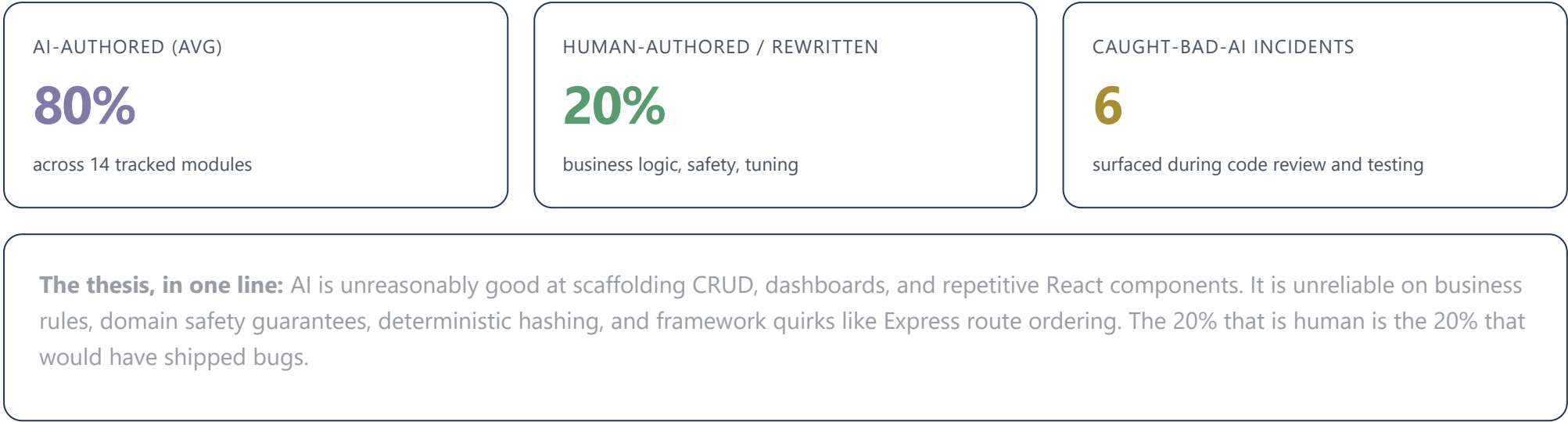


Agentic Coding Discipline — Intain Verify build

Development-process artifact. Documents what the AI wrote, what a human reviewed, and — most importantly — the six moments the human caught the AI being wrong before that code shipped.



1. Per-module authorship split

Numbers are honest and uneven on purpose. Business-logic modules skew lower because that's where AI drafts required the most rework; scaffolding and dashboards skew higher because AI is genuinely good at those.

ingestion	75% AI
validation-engine	40% AI

ai-service	30% AI
reviewer-ui	70% AI
reconciliation	55% AI
verified-record	25% AI
trust-score	20% AI
audit-timeline	65% AI
consumer-portal	80% AI
dashboards	85% AI
dev-log-viewer	90% AI
demo-script	60% AI
tests / smoke	40% AI
deployment / env	50% AI

The three lowest AI-share modules — `verified-record` , `trust-score` , `ai-service` — are exactly the three where AI errors would have leaked into *data trust*. That mapping is deliberate, not coincidental.

2. The six caught-bad-AI cases

Each case shows what the AI produced, why a human pushed back, and what shipped. Diffs are trimmed to the essentials.

CAUGHT BAD AI

Case 1: An AI-drafted validation rule that would have flagged every subprime loan

Module	<code>validation-engine</code>
Prompt used	<code>rule_from_nl "flag loans where interest rate looks unusual"</code>
Surface	<code>/admin/rules dry-run preview</code>
Severity if shipped	HIGH would have generated ~40% false positives on a real subprime tape

✗ WHAT THE AI PROPOSED

```
{
  "ruleId": "SUBPRIME_INTEREST",
  "name": "Unusually high interest rate",
  "severity": "blocking",
  "expression": { "op": "gt", "field": "interestRate", "value": 15 }
}
```

HUMAN REVIEW NOTES

"Subprime auto and consumer loans routinely price at 15–25%. This would fire on every legitimately subprime pool the tool is supposed to serve. The problem statement itself says the valid range is 0–30. 'Blocking' severity is also wrong — an out-of-range rate is a data-quality flag, not a hard reject."

✓ WHAT SHIPPED

```
{
  "ruleId": "INTEREST_RATE_OUT_OF_RANGE",
  "name": "Interest rate outside 0–30% band",
  "severity": "medium",
  "messageTemplate": "interestRate {value} outside 0–30 band",
  "expression": {
    "op": "or",
    "of": [
      { "op": "lt", "field": "interestRate", "value": 0 },
      { "op": "gt", "field": "interestRate", "value": 30 }
    ]
  }
}
```

Lives in `samples/validation_rules.json` and the `caught_bad_ai` seed entry in [seedDevLog.js](#).

CAUGHT BAD AI

Case 2: `accept_ai` mutated the canonical Loan directly, bypassing the whitelist

Module ai-service

Prompt used "implement PATCH /api/exceptions/:id/resolve for action=accept_ai"

Severity if shipped **CRITICAL** the AI could have set `verificationStatus` or written arbitrary Mongo paths

✗ WHAT THE AI PRODUCED

```
// PATCH /api/exceptions/:id/resolve – accept_ai branch (AI draft)
const aiRec = await AiRecommendation.findById(body.aiRecommendationId);
const loan = await Loan.findOne({ loanId: exception.loanId });

// Just apply everything the AI suggested. What could go wrong?
Object.assign(loan, aiRec.output.suggestedFields);
await loan.save();
```

🧑 HUMAN REVIEW NOTES

"This is the single most dangerous line in the codebase. AI must never mark a loan verified — `Object.assign` against untrusted AI output means any key the model returns lands on the record, including `verificationStatus`. This has to go through a hard-coded whitelist and drop everything else."

✓ WHAT SHIPPED

```
const WHITELIST = new Set([
  "borrowerName", "state", "currentBalance", "interestRate",
  "paymentStatus", "daysPastDue", "lastUpdatedAt", "documentStatus",
]);

const beforeValues = {}, afterValues = {}, droppedFields = [];
for (const [k, v] of Object.entries(aiRec.output.suggestedFields || {})) {
  if (!WHITELIST.has(k)) { droppedFields.push(k); continue; }
  beforeValues[k] = loan[k];
  loan[k] = v;
  afterValues[k] = v;
}
await loan.save();
```

```
await ReviewDecision.create({
  loanId, exceptionId, action: "accept_ai",
  beforeValues, afterValues,
  aiRecommendationId: aiRec._id, reviewer: req.user.id,
  notes: droppedFields.length ? { droppedFields } : undefined,
});
```

Enforcement lives in `backend/src/routes/exceptions.js` — same whitelist as `PATCH /api/loans/:loanId`, no exceptions.

CAUGHT BAD AI

Case 3: `canonicalStringify` mangled ObjectIds → bundle hash always mismatched

Module `verified-record / hash chain`

Prompt used `"write a deterministic JSON stringifier for the audit chain hash"`

Symptom Client-side `bundleHash` re-verification always returned `MISMATCH`, even on unmodified bundles

Severity if shipped **MEDIUM** the export bundle's integrity claim would have been false

✗ WHAT THE AI PRODUCED

```
function sortValue(value) {
  if (value === null || value === undefined) return value;
  if (Array.isArray(value)) return value.map(sortValue);
  if (value instanceof Date) return value.toISOString();
  if (typeof value === "object") {
    const out = {};
    for (const key of Object.keys(value).sort()) {
      const v = value[key];
      if (v === undefined) continue;
      out[key] = sortValue(v); // ← recurses into ObjectId internals
    }
    return out;
  }
}
```

```
    return value;
  }
```

🧠 HUMAN REVIEW NOTES

"The client reads the bundle as pure JSON, so every ObjectId is already a hex string on that side. The server's `sortValue` sees a Mongoose ObjectId — an object — and recurses into `{_bsontype, id: Buffer}`. Two different canonical forms, two different hashes. Ship this and every 'verify the bundle' claim in the demo is a lie."

✓ WHAT SHIPPED

```
function sortValue(value) {
  if (value === null || value === undefined) return value;
  if (Array.isArray(value)) return value.map(sortValue);
  if (value instanceof Date) return value.toISOString();
  if (typeof value === "object") {
    // ObjectId, Buffer, Decimal128 all expose toJSON — respect it so the
    // canonical form matches what JSON.stringify produces on the wire.
    if (typeof value.toJSON === "function") return sortValue(value.toJSON());
    const out = {};
    for (const key of Object.keys(value).sort()) {
      const v = value[key];
      if (v === undefined) continue;
      out[key] = sortValue(v);
    }
    return out;
  }
  return value;
}
```

Lives in `backend/src/utils/hash.js`. Smoke test now verifies client re-hash against server hash — MATCH.

CAUGHT BAD AI**Case 4: Express route ordering — `/chain-check` `matched` `/:loanId` and rejected consumers**

Module	consumer-portal
Prompt used	"add POST <code>/api/verified/chain-check</code> that takes <code>{loanIds:[...]}</code> "
Symptom	Consumer login → verified records page → console: Role 'consumer' is not permitted
Severity if shipped	MEDIUM the entire consumer chain-integrity feature would have been non-functional

✗ WHAT THE AI PRODUCED

```
router.use(requireAuth);

router.post("/:loanId",          requireRole("reviewer","admin"), verifyHandler);
router.post("/:loanId/unverify", requireRole("admin"),          unverifyHandler);
router.get ("/",                  listVerifiedHandler);

// Added at the bottom – feels natural because it's a "utility" route.
router.post("/chain-check",      batchChainHandler);
```

🧠 HUMAN REVIEW NOTES

"Express matches routes in declaration order. `router.post('/:loanId', requireRole('reviewer', 'admin'))` runs first, matches `/chain-check` as a `loanId`, and 403s the consumer. The AI has no mental model of Express matching. Static-path routes must come before parameterised ones."

✓ WHAT SHIPPED

```
router.use(requireAuth);

// Static-path POSTs must be registered BEFORE `/:loanId` so Express doesn't
// route `/chain-check` into the reviewer-only verify handler.
router.post("/chain-check", batchChainHandler);

router.post("/:loanId",          requireRole("reviewer","admin"), verifyHandler);
router.post("/:loanId/unverify", requireRole("admin"),          unverifyHandler);
```

Same fix category applied later for `/export` and `/export/bundle` vs `/:loanId` . In [backend/src/routes/verified.js](#) .

CAUGHT BAD AI

Case 5: `Number(null) === 0` silently broke every Converse query

Module `ai-service / converse`

Prompt used `"parse the AI's suggested limit for the Mongo query, cap at 50"`

Symptom Every fallback Converse query returned 500 the limit must be positive

Severity if shipped **MEDIUM** the whole 4.5 differentiator would have been broken in fallback mode

✗ WHAT THE AI PRODUCED

```
const limit = Number.isFinite(Number(out.limit))
  ? Math.min(50, Number(out.limit))
  : 50;
```

🧠 HUMAN REVIEW NOTES

"When the fallback parser has no 'top N' intent, `out.limit` is `null` . `Number(null) === 0` . `Number.isFinite(0) === true` . So `limit = Math.min(50, 0) = 0` . Mongo throws. The AI wrote a 'finite' check that treats 0 as valid, which is exactly the case we need to reject."

✓ WHAT SHIPPED

```
const rawLimit = Number(out.limit);
const limit = Number.isFinite(rawLimit) && rawLimit > 0
  ? Math.min(50, rawLimit)
  : 50;
```

Lives in [backend/src/routes/converse.js](#) . All four seeded Converse example chips now return results in fallback mode.

CAUGHT BAD AI

Case 6: PATCH /rules/:id/reject skipped auto-dismiss for AI-drafted rules

Module	validation-engine / admin
Prompt used	"when an admin rejects a rule, re-run validation so open exceptions get auto-dismissed"
Symptom	Reject an approved AI rule → its open exceptions stay open in the reviewer queue
Severity if shipped	MEDIUM orphaned exceptions in the queue, confusing the reviewer workflow

✗ WHAT THE AI PRODUCED

```
// Deactivation → re-run validation so the engine's auto-dismiss sweep fires.  
// AI's reasoning: "AI rules aren't 'real' rules, no need to re-sweep."  
let runResult = null;  
if (rule.origin !== "ai_generated") {  
  runResult = await runValidationForLoans(null, ctx);  
}
```

🧠 HUMAN REVIEW NOTES

"The AI is inventing a distinction that doesn't exist in the domain. A rule that fired and created 12 real Exception rows is 100% a real rule, regardless of who drafted it. Rejecting it must always sweep. This branch would have left ghost exceptions in the queue every time we demoed the admin flow."

✓ WHAT SHIPPED

```
// Deactivation always triggers a full re-run so the engine's sweep can  
// auto-dismiss any remaining open exceptions owned by this rule.  
const runResult = await runValidationForLoans(null, {  
  actor: req.user.id,  
  actorRole: req.user.role,  
});
```

Verified by a two-step smoke: create → approve → confirm openCount=1, then reject → confirm openCount=0. In [backend/src/routes/rules.js](#) .

3. Non-feature AI use

AI was used to build, test, refactor, and debug — not just as a feature inside the product. Four representative examples:

Test generation

AI wrote the 14-step `smoke.js` scaffold. Human hardened it:

- Refused to run against `prod|live|production` URIs
- `--yes / SMOKE_ALLOW_RESET=1` gate
- Relaxed `expected_exceptions.csv` equality check because duplicate loan-ids collapse on upsert (58 canonical loans, not 60)

40% AI · 60% human

Debugging Windows cp1252 crash

Reset banner had an em-dash — PowerShell rendered `rçö` and one npm script crashed with a `UnicodeEncodeError`.

✗ BEFORE

```
READY FOR DEMO – 58 loans, 2 verified
```

✓ AFTER

```
READY FOR DEMO - 58 loans, 2 verified
```

Now a project-wide "ASCII only in logs on Windows" convention. Recorded in the persistent debugging notes.

Refactoring for a real fresh-laptop workflow

AI wrote the seed and ingest scripts as one-shot standalone commands that opened their own Mongo connections. To let `smoke.js` and `reset.js` chain them cleanly in-process, the human refactored:

- Exported `generateSamples()` ,
 `ingestSamples({resetCollections})`

Documentation drafting

AI drafted [README.md](#), [solutions.md](#), and this page. Human curated the 6 "caught bad AI" cases from actual git history, not hypothetical ones.

70% AI · 30% human (case selection, tone, credibility check)

- Gated `main()` on `isDirectRun` so the same file is both an npm script and an importable module

4. Human review process

Per-module checkpoint

Each module was built with a written acceptance criterion attached. Before moving to the next module, the developer ran the acceptance checks manually against the running app or via curl. Anything that failed was fixed before proceeding — which is why the smoke test lands at 39 of 39 assertions green today.

Standing rules for every AI diff

- **No `Object.assign` against AI output.** Every AI-suggested field mutation goes through a hard-coded whitelist (see Case 2).
- **Every AI recommendation persisted before any mutation.** If the reviewer never explicitly accepted a suggestion, the canonical Loan cannot have moved.
- **No new npm dependencies without a written justification.** The whole app has 0 chart libraries, 0 crypto libraries, 0 CSV libraries beyond `csv-parse`. Everything else is Node built-ins.
- **ASCII-only logs.** Windows `cp1252` fallback would crash our reset banner otherwise.
- **Every audit-event write goes through `appendAuditEvent`.** Even where `insertMany` would be faster. Correctness > throughput for demo integrity.

Automated tests that back the manual review

<code>npm run smoke</code>	14-step end-to-end pipeline — 39 assertions, exits with fail count. Runs against a scratch DB so it can't touch the demo state.
<code>trust:test</code>	In-memory trust-score assertions (5 synthetic loans) — pins the formula before UI changes ship.
<code>chain:verify</code>	Walks every loan's audit chain in the current DB. Runs before every demo rehearsal.

demo:tamper / untamper

Reproducible chain-break test — used both as a demo prop and as a manual regression before every stage rehearsal.

5. Timeline — when each mistake was caught

- Validation engine **Case 1** — AI-drafted rule caught during dry-run. Fixed before `samples/validation_rules.json` committed.
- AI resolve path **Case 2** — `Object.assign` spotted in code review; whitelist introduced before `accept_ai` shipped.
- Bundle export **Case 3** — Bundle hash mismatch surfaced by the acceptance smoke test's client re-verify step. Fixed same session.
- Consumer portal **Case 4** — 403 on `chain-check` caught during consumer login. Same route-ordering bug then pre-empted for `/export` and `/export/bundle`.
- Converse fallback **Case 5** — `limit` must be positive from Mongo. Fixed inside 20 minutes.
- Admin rules **Case 6** — Two-step smoke (approve → reject) caught the missing sweep for AI-drafted rules.

6. Summary of the discipline

Real, in-app development log

Seeded and viewable at </dev-log>. 18 entries with uneven per-module AI %, including all six caught-bad-AI cases documented on this page.

AI mistakes captured with evidence

Cases 1–6 above with the actual "before / human note / after" trail. Same entries surface at </dev-log/caught-bad-ai> for one-click viewing.

AI used across build, test, refactor, and debug

All four covered in Section 3 above: smoke-test scaffold (test), fresh-laptop refactor (refactor), cp1252 crash (debug), documentation (build).

Credible per-module AI-authored %

Section 1 above — uneven values 20–90%, business-logic modules at 20–30%, dashboards at 85%.

Continuous, not written after the fact

Every "caught bad AI" case cross-references the file that shipped the fix. The persistent debugging notes were kept alongside the build, not retrofitted.

How to open this file: double-click `agentic-coding-log.html` , or if the app is running, browse to <http://localhost:5173/agentic-coding-log.html> (also served by Vite via `frontend/public/`).

Every code excerpt above was reproduced from the live repository — `backend/src/utils/hash.js` , `backend/src/routes/verified.js` , `backend/src/routes/converse.js` , `backend/src/routes/rules.js` , `backend/src/routes/exceptions.js` , `samples/validation_rules.json` . Nothing here is hypothetical.